Annex F

BPMAPPER.M Program Code (*Matlab 4.0*)

```
function bpmapper(FARTMAPFile)
% BPMapper - Back Propogation Neural Network
%
% This program inputs the Fuzzy ART Map LTM traces (W Matrix) as training vectors and
% the Signal Identification Matix (SIM) as target vectors.
% This network utilizes Nguyen-Widrow initial conditions, adaptive learning rates and
% momentum with the back-propogation learning rule in an attempt to map the inputs to the
% desired outputs.

help bpmapper

% Load input and target vectors
load(FARTMAPFile);
% where W is the weight file created by FARTMAPT.M which is already normalized to
% the fuzzy values and includes ones complement coding and where SIM is the Signal
% Identification Matrix created by FARTMAPT.M


% remove the ones complement component to reduce input vector length
[Wrows,Wcols]=size(W);
P=W(:,1:Wcols/2);
%convert to column vectors for compatibility with the BP algorithm below
P=P';

% Convert target vectors from text to numerical format (local binary representation)
[SIMrows,SIMcols]=size(SIM);
```

```
T=zeros(SIMrows,8);

for rownum=1:SIMrows

        PHRASE=(SIM(rownum,1:5));

           if PHRASE=='SHE H' T(rownum,:)=[1 0 0 0 0 0 0 0];
        elseif PHRASE=='DON^T' T(rownum,:)=[0 1 0 0 0 0 0 0];
        elseif PHRASE=='MAY W' T(rownum,:)=[0 0 1 0 0 0 0 0];
        elseif PHRASE=='WE WE' T(rownum,:)=[0 0 0 1 0 0 0 0];
        elseif PHRASE=='I KNO' T(rownum,:)=[0 0 0 0 1 0 0 0];
        elseif PHRASE=='EVERY' T(rownum,:)=[0 0 0 0 0 1 0 0];
        elseif PHRASE=='OUR Y' T(rownum,:)=[0 0 0 0 0 0 1 0];
        elseif PHRASE=='I WAS' T(rownum,:)=[0 0 0 0 0 0 0 1];
        else   T(rownum,:)=[0 0 0 0 0 0 0 0]; % in case of blank entries in the SIM Matrix
        end; %if PHRASE
end;

% convert T to column vector format
T=T';

% INITIALIZE NETWORK ARCHITECTURE
%================================
% Set input vector size R, layer sizes S1 & S2, batch size Q.
[R,Q] = size(P);
S1 = 50;
[S2,Q] = size(T);

% Initialize weights and biases.
%z = menu('Initialize Weights & Bias To', ...
%  'Stored Values in BPMAPPER.MAT', ...
```

```
%  'Random Nguyen-Widrow Values [Default]');
%disp('')
%if z == 1
        load('bpinit.mat');
%else
%       [W10,B10] = nwtan(S1,R);
%       W20 = rands(S2,S1)*0.5;
%       B20 = rands(S2,1)*0.5;
% save bpinit.mat W10 W20 B10 B20
%end


% TRAIN THE NETWORK
%==================

% TRAINING PARAMETERS

df = 10;   % Display Frequency
me = 900; % Maximum Epochs to calculate
eg = 0.1; % Error Goal
lr = 0.0001; % Learning Rate
im = 1.05; % learning rate IncreMent
dm = 0.7;  % learning rate DecreMent
mc = 0.95; % Momentum Constant
er = 1.04; % Error Ratio

% NETWORK PARAMETERS

load('bprtmapt.mat');
%W1 = W10;
%B1 = B10;
```

```
%W2 = W20;
%B2 = B20;

dW1 = W1*0;
dB1 = B1*0;
dW2 = W2*0;
dB2 = B2*0;
MC = 0;

% PRESENTATION PHASE

A1 = tansig(W1*P,B1);
A2 = purelin(W2*A1,B2);
E = T-A2;
SSE = sumsqr(E);

% TRAINING RECORD
TR = zeros(2,me);
SSE0 = SSE;
lr0 = lr;

% BACKPROPAGATION PHASE
D2 = deltalin(A2,E);
D1 = deltatan(A1,D2,W2);

figure(1)        % Create a new figure to plot the learning process (and print later)
clf reset
hold on

for epoch=1:me
```

```
% CHECK PHASE

if SSE < eg
        epoch=epoch-1;
        break;
end;

% LEARNING PHASE
[dW1,dB1] = learnbpm(P,D1,lr,MC,dW1,dB1);
[dW2,dB2] = learnbpm(A1,D2,lr,MC,dW2,dB2);
MC = mc;
TW1 = W1 + dW1; TB1 = B1 + dB1;
TW2 = W2 + dW2; TB2 = B2 + dB2;

% PRESENTATION PHASE
TA1 = tansig(TW1*P,TB1);
TA2 = purelin(TW2*TA1,TB2);
TE = T-TA2;
TSSE = sumsqr(TE);

% MOMENTUM & ADAPTIVE LEARNING RATE PHASE
if TSSE > SSE*er
  lr = lr * dm;
  MC = 0;
else
  if TSSE < SSE
    lr = lr * im;
  end

  W1=TW1; B1=TB1; W2=TW2; B2=TB2;
  A1=TA1; A2=TA2; E=TE; SSE=TSSE;
```

```
    % BACKPROPAGATION PHASE
    D2 = deltalin(A2,E);
    D1 = deltatan(A1,D2,W2);
  end

  % TRAINING RECORD
  TR(1,epoch) = SSE;
  TR(2,epoch) = lr;

  % DISPLAY RESULTS
  if rem(epoch,df) == 0
    plottr([[SSE0; lr0] TR(:,1:epoch)], ...
      '2-Layer Backpropagation with Adaptive LR & Momentum')
  end
  fprintf('\nEPOCH=%i SSE=%f LR=%f \n',epoch,SSE,lr);
end


% RETURN RESULTS
TR = [[SSE0; lr0] TR(:,1:epoch)];


% PLOT LEARNING RATES
%====================
figure(2)
clf reset
hold on

plotlr(TR(2,:));
pause(3)
```

```
% PLOT ERROR CURVE
%================
figure(3)
clf reset
hold on

ploterr(TR(1,:));
pause(3)

% PLOT ERRORS ASSOCIATED WITH EACH OUTPUT VECTOR
%===============================================
figure(4)
clf reset
hold on

barerr(A2,T);
pause(3)

% SUMMARIZE RESULTS
%==================
SSE = sumsqr(T-purelin(W2*tansig(W1*P,B1),B2));

diary trgdiary.txt
fprintf('Trained for %.0f epochs.\n',epoch)
fprintf('Training took %.0f flops.\n',flops);
fprintf('Sum squared error goal was %g.\n',eg);
fprintf('Final sum squared error is %g.\n',SSE);
diary off

fprintf('\nSAVING FINAL NETWORK VALUES:\n')
```

```
save tempfile W1 B1 W2 B2 FARTMAPFile
clear
load tempfile
save(['bp',FARTMAPFile(3:length(FARTMAPFile)-4),'.mat'])
```